Lab 8

Synchronization

Course: Operating System **Class: AI Section A and B**

Instructor: Bilal Ahmed

Learning Objectives:

- Learn about synchronization and its importance.
- Creating and solving race condition.
- Understand and implement mutex and seamaphore.
- Calculating the mutex and semaphore overhead.

Synchronization in operating systems refers to the coordination of concurrent processes or threads to ensure that they behave correctly when accessing shared resources such as memory, files, or hardware devices. It's necessary because in a multitasking environment, multiple processes or threads may run concurrently and access shared resources simultaneously. Without proper synchronization, several issues can arise:

1. **Race Conditions**: When multiple processes or threads access shared resources simultaneously, the outcome of the execution may depend on the timing of their execution. This can lead to unpredictable behavior and erroneous results.
2. **Data Inconsistency**: If multiple processes or threads are modifying shared data concurrently, it may result in data becoming inconsistent or corrupted. For example, if one thread is updating a variable while another thread is reading it, the reader may see an intermediate or invalid value.
3. **Deadlocks**: Deadlocks occur when two or more processes or threads are waiting indefinitely for each other to release resources that they need. Proper synchronization mechanisms help avoid deadlocks by ensuring that processes release resources in a coordinated manner.
4. **Starvation**: Starvation happens when a process is perpetually denied access to a resource it needs due to other processes continually taking precedence. Synchronization mechanisms help prevent starvation by providing fairness in resource allocation.

To address these issues, operating systems provide various synchronization primitives such as locks, semaphores, monitors, and condition variables. These primitives allow developers to coordinate the access to shared resources and ensure that concurrent processes or threads execute safely and efficiently.

C Code for Creating race condition:

```c
#include <stdio.h>
#include <pthread.h>
#define NUM_THREADS 2
#define NUM_INCREMENTS 1000000
// Shared variable
int counter = 0;
// Function executed by each thread to increment the counter
void *increment(void *arg) {
    for (int i = 0; i < NUM_INCREMENTS; i++) {
        counter++; // Increment the counter
    }
    pthread_exit(NULL);
}
int main() {
    pthread_t threads[NUM_THREADS];
    // Create threads
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }
    // Wait for threads to finish
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    // Display the final value of the counter
    printf("Final counter value: %d\n", counter);
    return 0;
}
```

Output:

Explanation for incorrect value:

- **Concurrency**: Two threads (thread1 and thread2) concurrently increment the shared variable counter.
- **Race Condition**: Unsynced access to counter by both threads may lead to unpredictable results due to overlapping operations.
- **Non-Atomic Operation**: Incrementing counter involves multiple steps and isn't atomic, allowing interference between threads.
- **Interleaved Execution**: Operating system scheduler interleaves thread execution, influencing the order of operations.
- **Potential Overwrites**: If one thread reads counter while the other is modifying it, overwrites may occur, causing lost updates.
- **Final Value Limitation**: Without synchronization, the final value of counter cannot exceed 2,000,000, as each thread increments it 1,000,000 times.
- **Example:** If two threads attempt to increment the counter simultaneously and both read its initial value (let's say 0), they may both increment it to 1 and then write it back. In this case, the final value of the counter would be 1 instead of 2 (as it should have been if the increments were done sequentially).

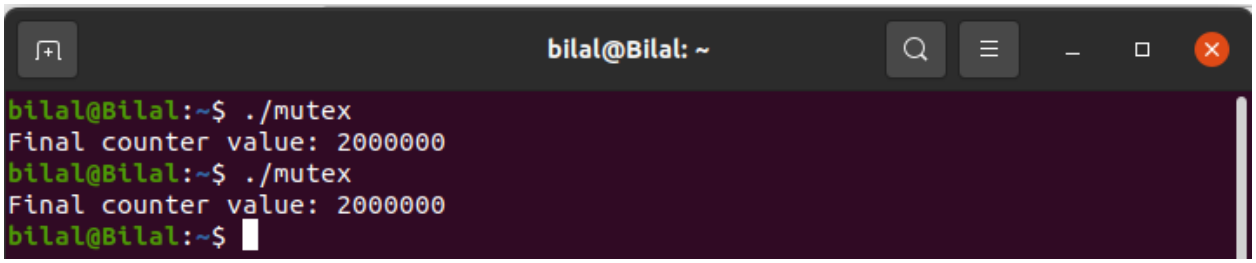
Mutual Exclusion: Mutual exclusion, as facilitated by a mutex lock, is a fundamental synchronization principle in concurrent programming. It guarantees that at any given time, only one thread can access a shared resource or a critical section of code. When a thread acquires a mutex lock, it gains exclusive access to the protected resource, effectively preventing other threads from accessing it concurrently. This ensures that conflicting operations do not occur simultaneously, thereby avoiding race conditions and preserving data integrity. Mutex locks provide a reliable means of enforcing mutual exclusion, allowing multiple threads to safely access shared resources in a serialized manner.

Mutex Lock: We can use pthread library to implement mutex lock mechanism as given in below C Code.

C code for implementing mutex.

```c
#include <stdio.h>
#include <pthread.h>
#define NUM_THREADS 2
#define NUM_INCREMENTS 1000000
// Shared variable
int counter = 0;
// Mutex lock
pthread_mutex_t lock;
// Function executed by each thread to increment the counter
void *increment(void *arg) {
    for (int i = 0; i < NUM_INCREMENTS; i++) {
        // Lock the mutex before accessing the counter
        pthread_mutex_lock(&lock);
        counter++; // Increment the counter
        // Unlock the mutex after accessing the counter
        pthread_mutex_unlock(&lock);
    }
    pthread_exit(NULL);
}
int main() {
    pthread_t threads[NUM_THREADS];
    // Initialize the mutex lock
    pthread_mutex_init(&lock, NULL);
    // Create threads
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }
    // Wait for threads to finish
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);

    printf("Final counter value: %d\n", counter);

    return 0;
}
```

Output:



Explanation:

- **Mutex Initialization**: A mutex lock (pthread_mutex_t) was initialized before the threads were created using pthread_mutex_init.
- **Mutex Locking**: Before accessing or modifying the shared variable counter, each thread locks the mutex using pthread_mutex_lock(&lock).
- This ensures that only one thread can access the critical section (the code block where counter is modified) at a time.
- **Mutex Unlocking**: After completing the critical section, each thread unlocks the mutex using pthread_mutex_unlock(&lock), allowing other threads to access the critical section.
- **Mutual Exclusion**: By using the mutex lock, mutual exclusion is enforced, ensuring that only one thread can modify the shared variable counter at any given time.
- **Synchronization**: The mutex lock serves as a synchronization mechanism, coordinating access to the shared resource (counter) among multiple threads.

Mutex Overhead: Mutex overhead refers to the additional computational cost incurred by using mutexes for synchronization in multithreaded programs. This overhead arises due to several factors:

- **Acquiring and Releasing**: Mutexes require acquiring and releasing operations to lock and unlock access to shared resources. These operations involve system calls and synchronization mechanisms (e.g., atomic operations), which introduce overhead in terms of CPU cycles and potentially context switches.
- **Contention**: In scenarios where multiple threads contend for the same mutex (i.e., multiple threads attempt to acquire the mutex concurrently), contention can

occur. Contention leads to threads waiting for the mutex to become available, resulting in delays and potential inefficiencies.

- **Resource Utilization**: Mutexes consume system resources such as memory for data structures used to manage them and kernel resources for synchronization primitives. While these resources are typically minimal, they can become significant in highly concurrent applications or on resource-constrained systems.

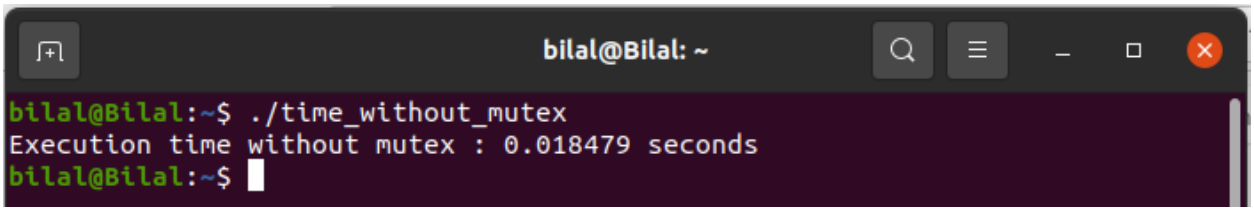
Calculating Times: Now we will calculate and compare the time taken by the programs; with and without mutex.

C code without mutex.

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
#define NUM_THREADS 2
#define NUM_INCREMENTS 1000000

int counter = 0;
void *increment(void *arg) {
   for (int i = 0; i < NUM_INCREMENTS; i++) {
      counter++; // Increment the counter without synchronization
   }
   pthread_exit(NULL);
}
int main() {
   pthread_t threads[NUM_THREADS];

   clock_t start_without_mutex_and_threads = clock();

   for (int i = 0; i < NUM_THREADS; i++) {
      pthread_create(&threads[i], NULL, increment, NULL);
   }
   for (int i = 0; i < NUM_THREADS; i++) {
      pthread_join(threads[i], NULL);
   }
   clock_t end_without_mutex_and_threads = clock();

   double time_without_mutex_and_threads = ((double)(end_without_mutex_and_threads -
start_without_mutex_and_threads)) / CLOCKS_PER_SEC;

   printf("Execution time without mutex : %f seconds\n", time_without_mutex_and_threads);
```

```
    return 0;
}
```

Output

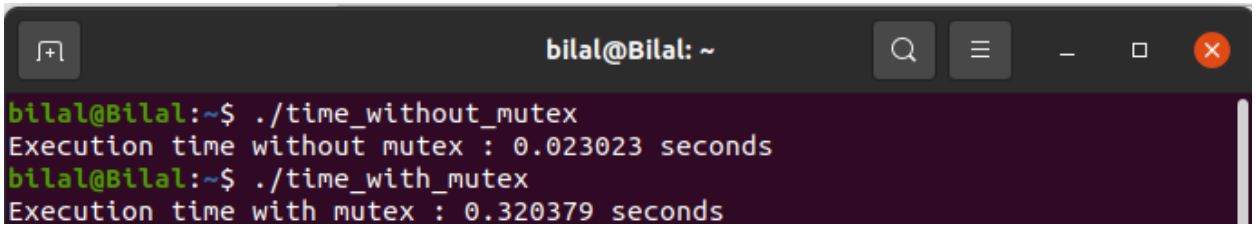


C code with mutex.

```c
#include <stdio.h>
#include <pthread.h>
#include <time.h>

#define NUM_THREADS 2
#define NUM_INCREMENTS 1000000
int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg) {
    for (int i = 0; i < NUM_INCREMENTS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    pthread_exit(NULL);
}
int main() {
    pthread_t threads[NUM_THREADS];
    pthread_mutex_init(&lock, NULL);

    clock_t start_with_mutex = clock();

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    clock_t end_with_mutex = clock();

    double time_with_mutex = ((double)(end_with_mutex - start_with_mutex)) / CLOCKS_PER_SEC;
```

```
    printf("Execution time with mutex : %f seconds\n", time_with_mutex);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:



Semaphore. A semaphore is a synchronization tool used in concurrent programming to regulate access to shared resources among multiple threads or processes. It controls access by maintaining a counter and supporting two main operations: wait (P) and signal (V).

C code for multiple threads without semaphore.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

int counter = 0;
sem_t sem;

void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        sem_wait(&sem);   // LOCK
        counter++;
        sem_post(&sem);   // UNLOCK

    }
    return NULL;
}
int main() {
    pthread_t t1, t2;
    sem_init(&sem, 0, 1);
```

```
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter = %d\n", counter);

    sem_destroy(&sem);
    return 0;
}
```

Semaphore with multiple resources.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define THREADS 6

sem_t semaphore;  // controls max 3 connections

void* queryDatabase(void* arg) {
    char* threadName = (char*)arg;

    printf("%s waiting for connection...\n", threadName);

    sem_wait(&semaphore);  // acquire (block if 0)

    printf("%s got connection, querying...\n", threadName);
    sleep(2);  // simulate DB work

    sem_post(&semaphore);  // release
    printf("%s released connection\n", threadName);

    return NULL;
}

int main() {
    pthread_t threads[THREADS];
    char names[THREADS][20];

    // Initialize semaphore with 3 permits
    sem_init(&semaphore, 0, 3);
```

```
    // Create 6 threads
    for (int i = 0; i < THREADS; i++) {
        sprintf(names[i], "Thread-%d", i + 1);
        pthread_create(&threads[i], NULL, queryDatabase, names[i]);
    }

    // Wait for all threads to finish
    for (int i = 0; i < THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    // Destroy semaphore
    sem_destroy(&semaphore);

    return 0;
}
```

Output:

Exercise:

Task 1: Implement the producer-consumer problem using mutex.

Task 2: Write a single code which implements increment problem in both ways (with and without mutex) and calculate the time overhead at the end of the code.

Task 3: Calculate the time overhead of semaphore implemented above.